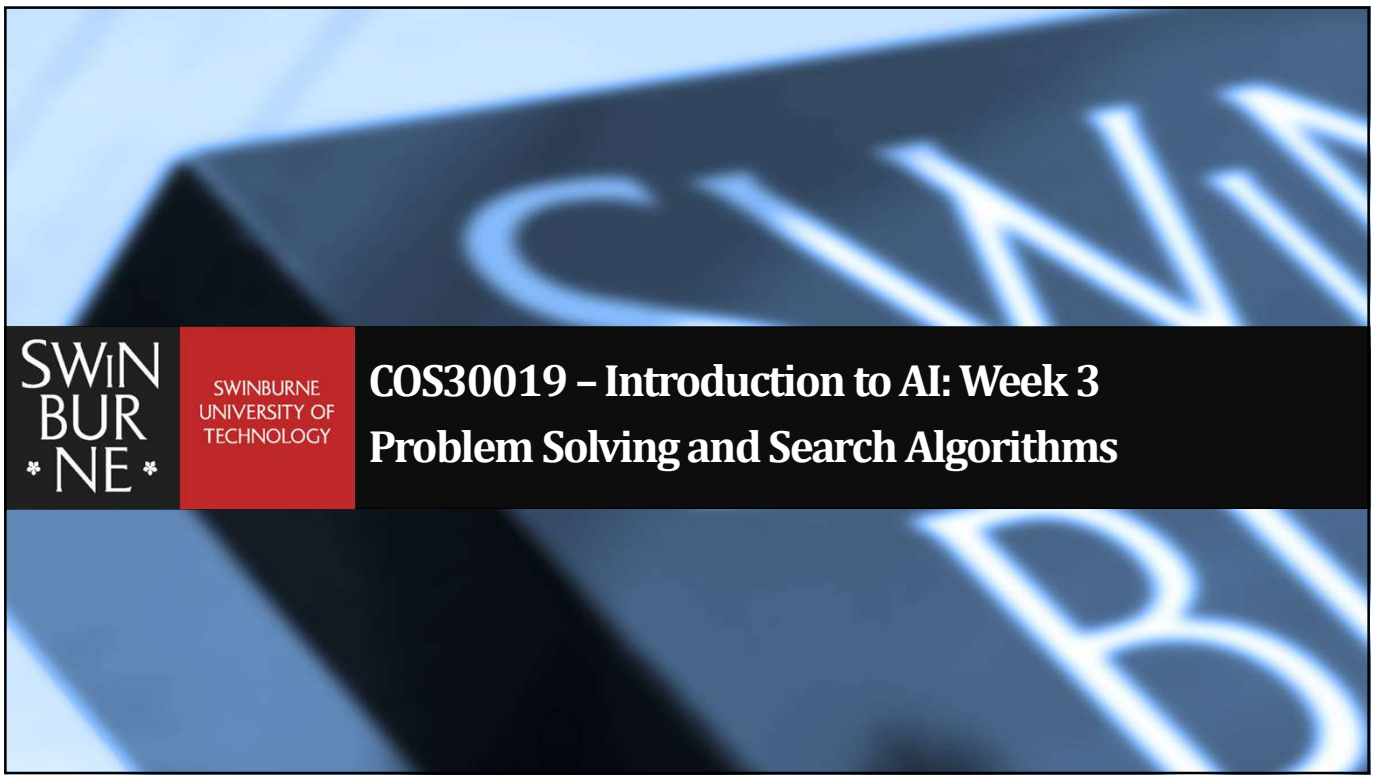


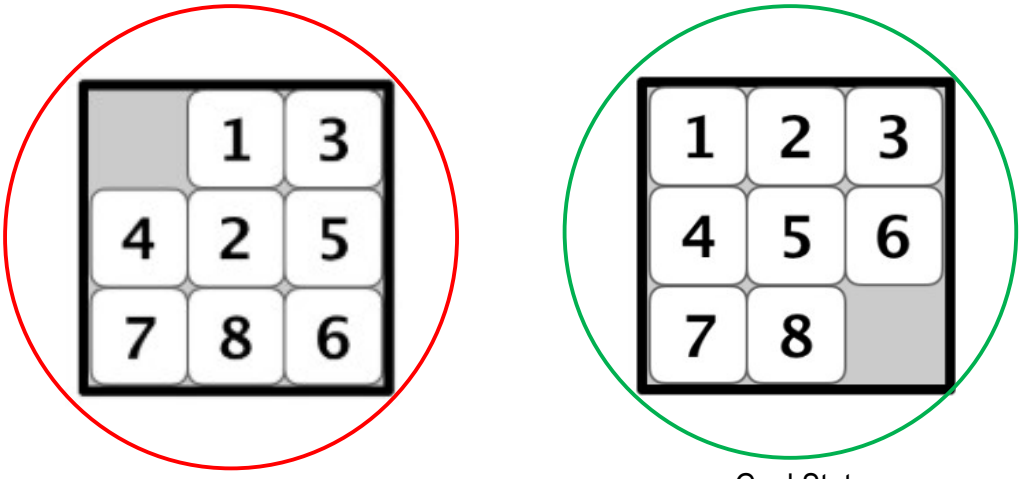
1

Overview

- Problem-solving agents
  - A kind of goal-based agent
- Problem types
  - Single state (fully observable)
- Problem formulation
  - Example problems
- Basic search algorithms
  - Uninformed

2

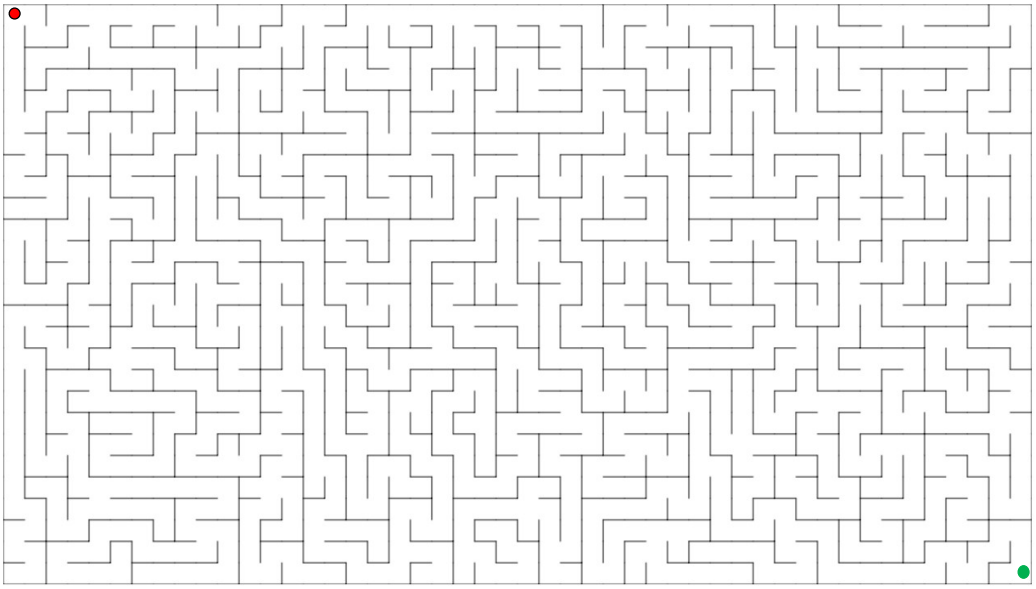
Example: 8-puzzle



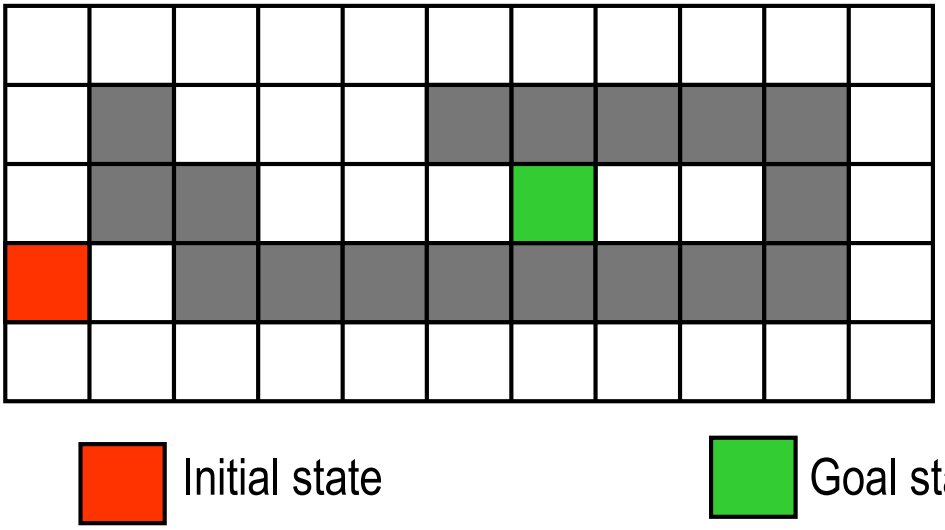
Initial State
(aka. Start State)

Goal State

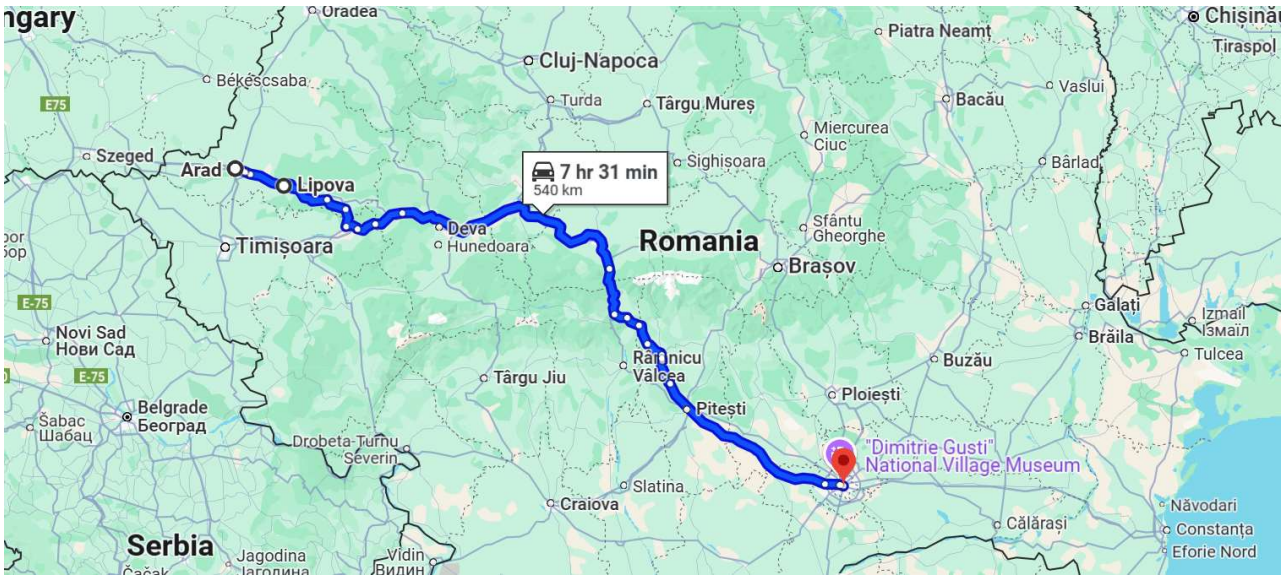
Example: Solving a maze



Example: Robot Navigation



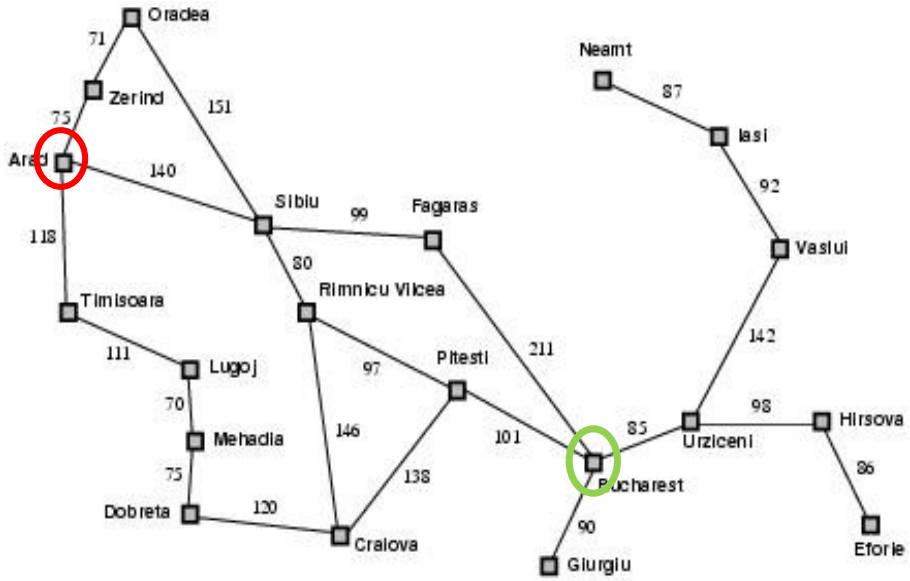
Example: Route finding



State

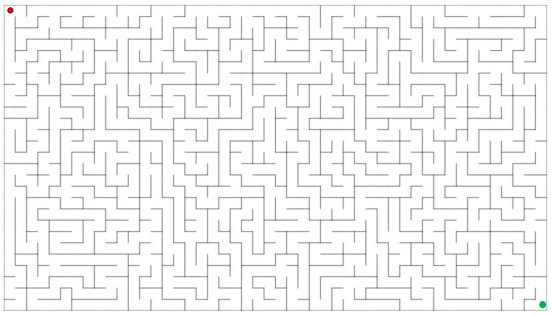
- A configuration of the agent and its environment

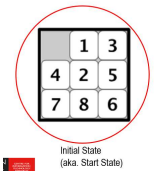
Example: Route finding

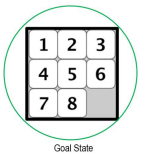


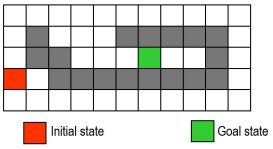
Initial state

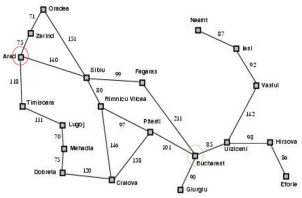
- A state in which the agent begins
- i.e., the agent solves the problem by finding the sequence of **actions** that leads it from the *initial state* to a goal state











SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Actions

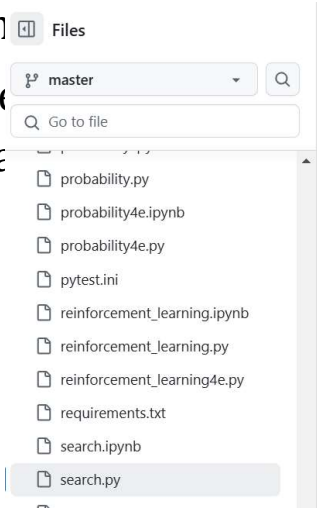
- Choices that can be made in a state (because the agent can perform them); aka. *decisions*.
  - Given a state *s*, *Actions*(*s*) returns a set of actions/choices the agent can take in state *s*

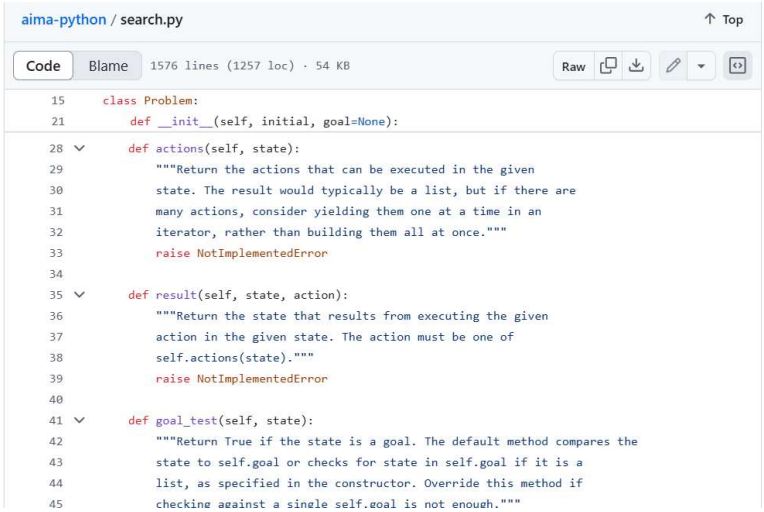
SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Actions

•Choices that can be made in a state (because the agent can perform

•Give
the a





SWIN BUR NE | CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Transition model

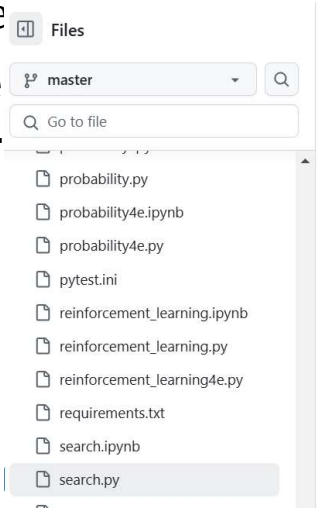
•A description of the resulting state when executing an action *a* in a state *s*.

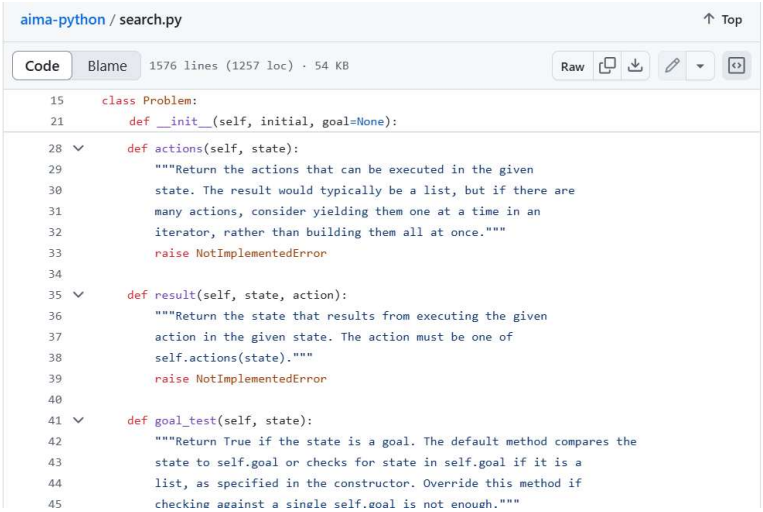
•Given a state *s* and an applicable action *a*, *Result*(*s, a*) returns *the state* resulting from executing *a* in state *s.*

SWIN BUR NE | CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

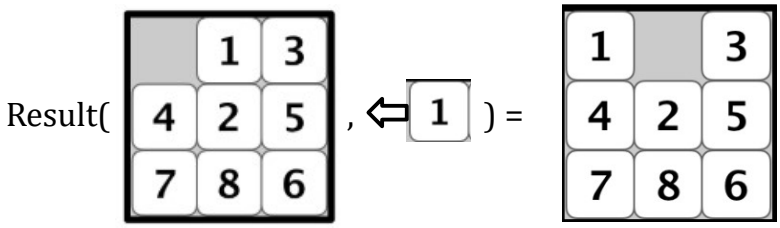
Transition model

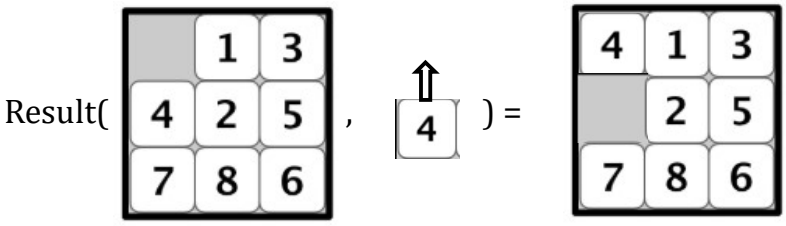
- A description of the resulting state when executing an action *a* in a state
  - Give
  - retur



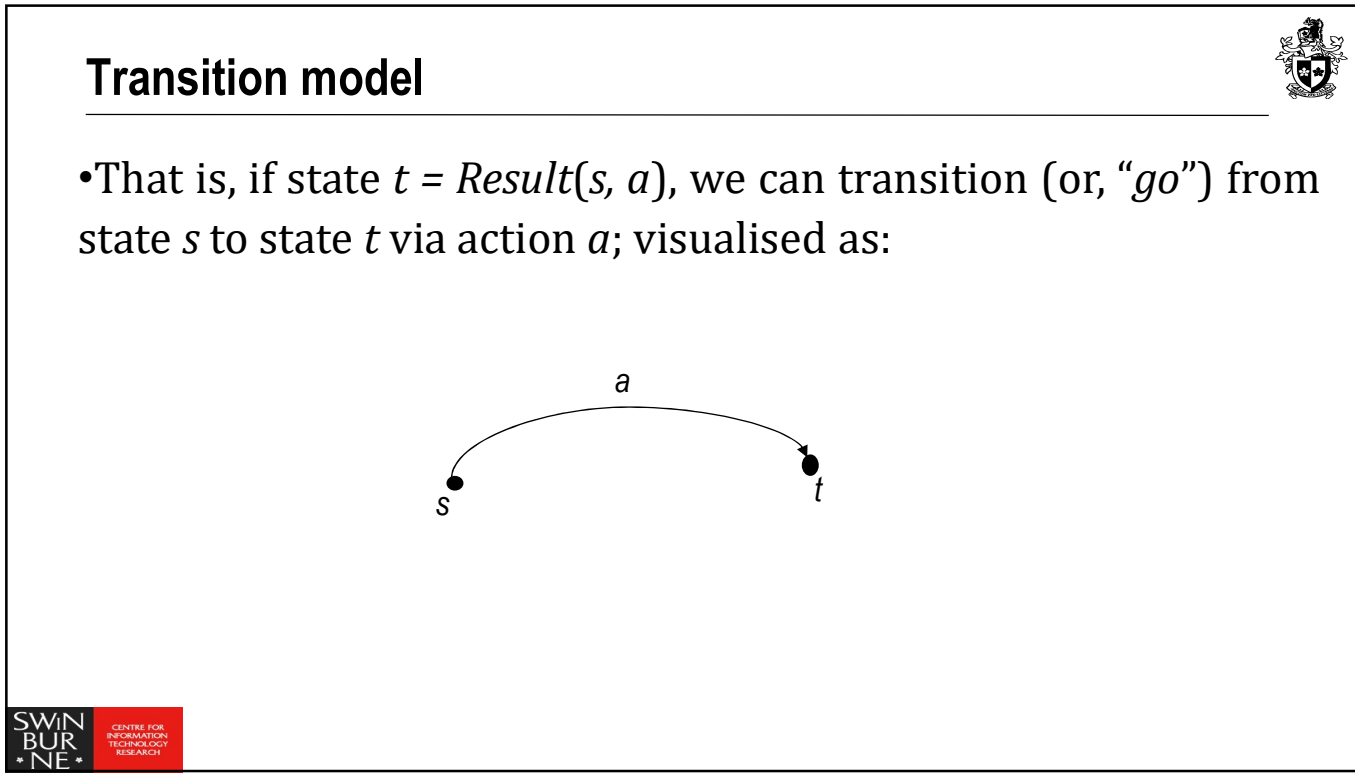


Transition model - Example



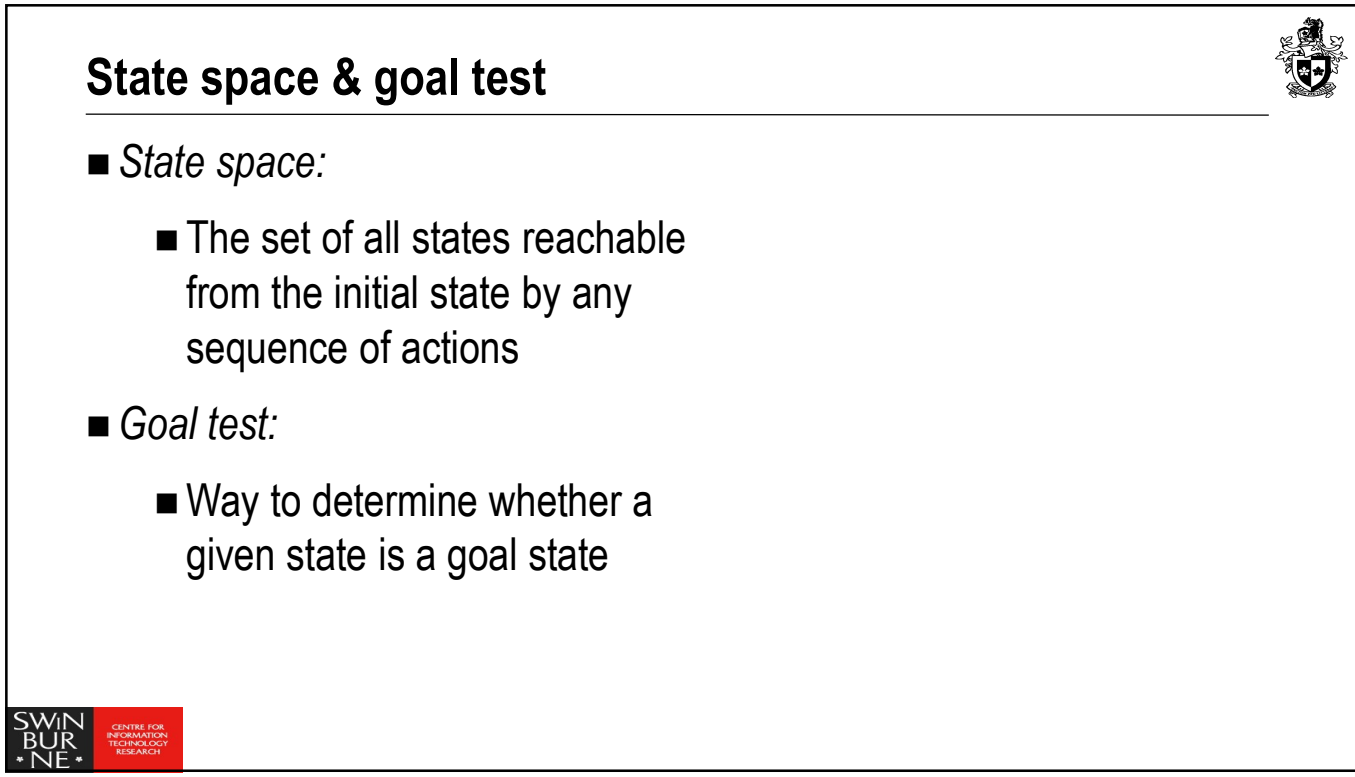


Transition model



•That is, if state $t = Result(s, a)$, we can transition (or, "*go*") from state s to state t via action a; visualised as:

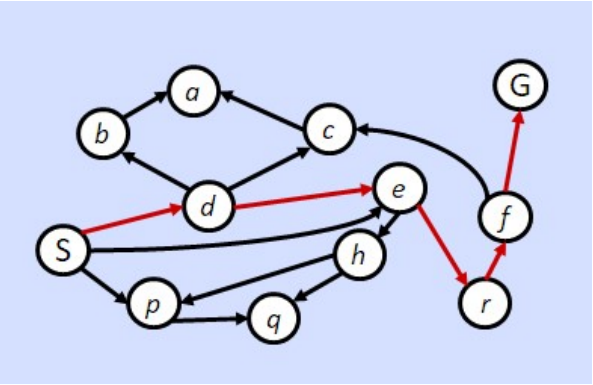
State space & goal test



- *State space:*
  - The set of all states reachable from the initial state by any sequence of actions
- *Goal test:*
  - Way to determine whether a given state is a goal state

State space & goal test

- *State space:*
  - The set of all states reachable from the initial state by any sequence of actions
- *Goal test:*
  - Way to determine whether a given state is a goal state



Goal test = Agent is in state G

State space & goal test

- *State space:*
  - The set of all states reachable from the initial state by any sequence of actions
- *Goal test:*
  - Way t
  - given

Initial state = A

- But perhaps:
  - **In C**: Agent having $10M and being wanted by the police for bank robbery
  - **In P**: Agent having $10M after many years of hard work and saving and investing

Goal test = Agent has $10M

Problem-solving agent

- Four general steps in problem solving:
  - Goal formulation
    - What are the successful world states → Goal test?
  - Problem formulation
    - What actions and states to consider given the goal → State space?
  - Search
    - Determine the possible sequence of actions that lead to the states of known values and then choosing the best sequence. (i.e., solution = the best sequence.)
  - Execute
    - Given the solution, perform the actions.

Problem-solving agent

- Four general steps in problem solving:
  - Goal formulation
    - What are the successful world states → Goal test?
  - Problen
    - Wh
  - Search
    - Det
      and
  - Execute
    - Given the solution, perform the actions.

- (Solution ~) The best sequence??
  - One with the lowest path cost.
- Path = sequence of actions
- Path cost??
  - Numerical cost associated with a given path

Search problem (single-state)

- A **problem** is defined by the following:

1. initial state e.g., "at Arad"
2. actions
3. transition model (i.e., the *Result*(*s*,*a*) function)
4. goal test; can be
   - explicit, e.g., *x* = "at Bucharest"
   - implicit, e.g., *Checkmate(x)*
5. path cost:
   - Typically additive; e.g., sum of distances, number of actions executed, etc.
   - *c(s, a, t)* is the step cost (for the step of taking action *a* to transition from state *s* to state *t*; assumed to be ≥ 0

- A **solution** is a sequence of actions leading from the initial state to a goal state

Search problem (single-state)

- A **problem** is def

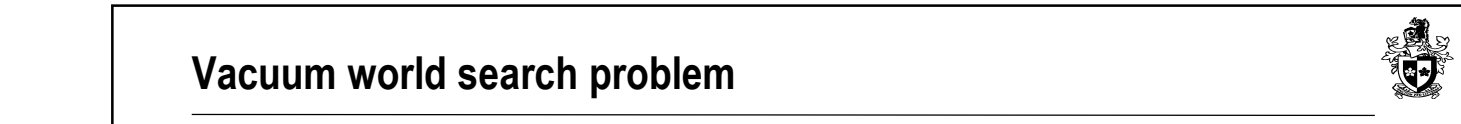
1. initial state e.g., '
2. actions
3. transition model
4. goal test; can be
   - explicit, e.g.,
   - implicit, e.g.,
5. path cost:
   - Typically add
   - *c(s, a, t)* is the step cost (for the step of taking action *a* to transition from state *s* to state *t*; assumed to be ≥ 0

Transition model

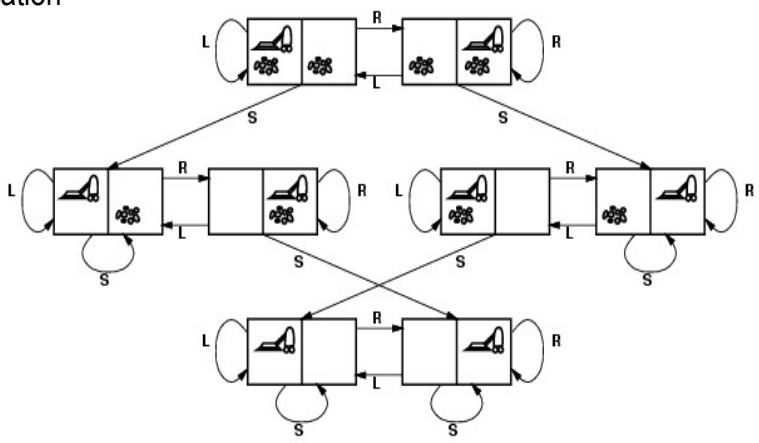
- A description of the resulting state when executing an action *a* in a state *s*.
  - Given a state *s* and an applicable action *a*, *Result*(*s*, *a*) returns *the state* resulting from executing *a* in state *s*.

- A **solution** is a sequence of actions leading from the initial state to a goal state

Vacuum world search problem



- states? integer dirt and robot location
- actions? *Left*, *Right*, *Suck*



- goal test? no dirt at all locations
- path cost? 1 per action

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Problem types

- Deterministic, fully observable ⇒ *single state problem*
  - Agent knows exactly which state it will be in; solution is a sequence.
- Partial knowledge of states and actions:
  - Non-observable ⇒ *sensorless or conformant problem*
    - Agent may have no idea where it is; solution (if any) is a sequence.
  - Nondeterministic and/or partially observable ⇒ *contingency problem*
    - Percepts provide *new* information about current state; solution is a tree or policy; often interleave search and execution.
  - Unknown state space ⇒ *exploration problem* (“online”)
    - When states and actions of the environment are unknown.

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Solving problems using search algorithms

Basic search algorithms – An overview

- How do we find the solutions of previous problems?
  - Search the **state space** (remember complexity of space depends on state representation)
  - Here: search through *explicit tree generation*
    - ROOT= initial state.
    - Nodes are generated through **transition model** (i.e., the *Result*(*s, a*) function)
  - In general search generates a graph (same state through multiple paths)

Basic search algorithms – An overview

- How do we find the solutions of previous problems?
  - Search the **state space** (remember complexity of space depends on state representation)
  - Here: search through *explicit tree generation*
    - ROOT= initial state.
    - Nodes are generated through **transition model** (i.e., the *Result*(*s, a*) function)
  - In gene

- If this slide makes sense to you, you're good and can go having a cup of coffee and skip the next 10 minutes of this lecture.
- If it does not, stay tuned.

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

The intuition behind search algorithms

- In any given state *s*:
  - By considering all actions applicable in *s*, we can see the available options from *s*
    - An option is a state reachable from *s* via one of the applicable actions
    - This step is called **exploring** *s* or sometimes **expanding** *s*.
  - Repeating the process by exploring those available options, we'll continue to see even more options
  - As more and more options are being discovered and some of them have been expanded while others are still available for expansion, we need a good data structure to organize them
    - The set of options that have not been expanded will be stored in a single data structure called **FRONTIER** to allow the search algorithm to consider them when it need to expand an option.

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

The intuition behind search algorithms

- In any given state *s*:
  - By considering all actions applicable in *s*, we can see the available options from *s*
    - An option is a state reachable from *s* via one of the applicable actions
    - This step is called **exploring** *s* or sometimes **expanding** *s*.
  - Repeating the process by exploring those available options, we'll continue to see even more options
  - As more and more o… expanded while othe… …cture to organize them
    - The set of options that have not been expanded will be stored in a single data structure called **FRONTIER** to allow the search algorithm to consider them when it need to expand an option.

The **FRONTIER** represents all options that have not been explored before and that we can explore next.

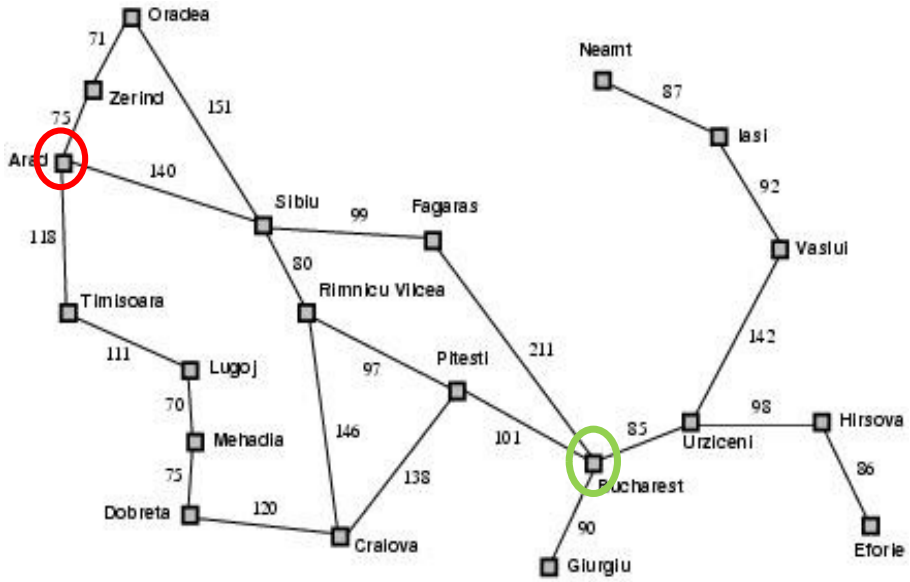
SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Search algorithms - informally

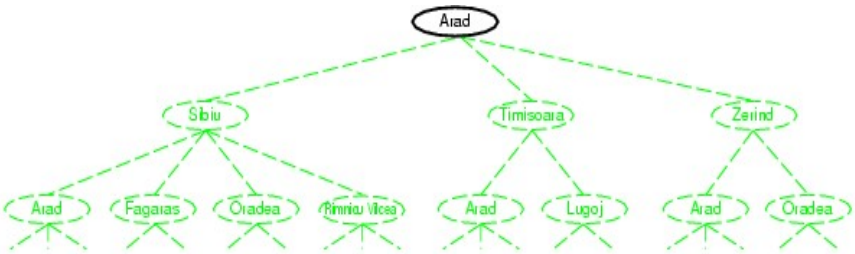
- Starts with a **FRONTIER** that contains the initial state
- Repeat:
  - If the **FRONTIER** happens to be empty, then *no solution*
  - Take a *node* out of the **FRONTIER** (this is *the option we choose to expand next*)
  - If that *node* contains a goal state, return the *found solution*
  - Else, expand *node*, add the resulting nodes (from *node*) to the **FRONTIER**

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Example: Route finding



Simple tree search example



function TREE-SEARCH(*problem, frontier*) **return** a solution or failure

frontier ← INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

loop do

if EMPTY?(*frontier*) **then return** failure

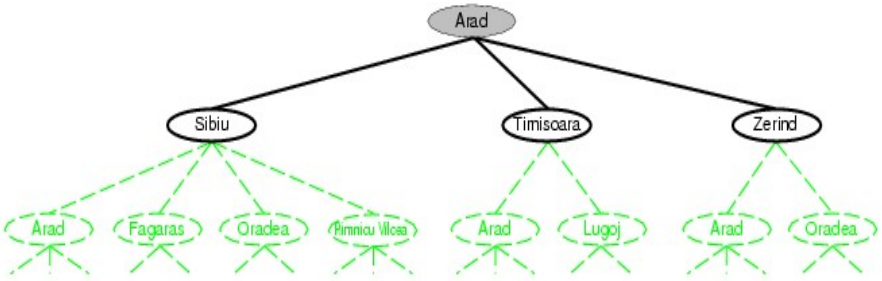
node ← REMOVE-FIRST(*frontier*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

frontier ← INSERT-ALL(EXPAND(*node*, *problem*), *frontier*)

Simple tree search example



function TREE-SEARCH(*problem, frontier*) **return** a solution or failure

frontier ← INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

loop do

if EMPTY?(*frontier*) **then return** failure

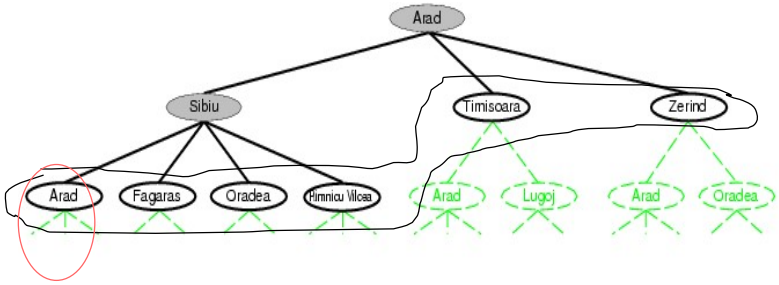
node ← REMOVE-FIRST(*frontier*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

frontier ← INSERT-ALL(EXPAND(*node*, *problem*), *frontier*)

Simple tree search example



function TREE-SEARCH(*problem, frontier*) **return** a solution or failure

frontier ← INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

loop do

if EMPTY?(*frontier*) **then return** failure

node ← REMOVE-FIRST(*frontier*)

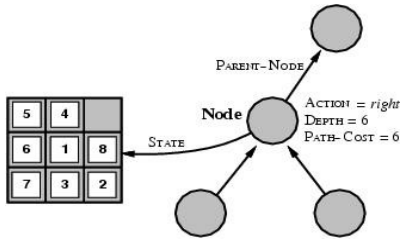
if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

frontier ← INSERT-ALL(EXPAND(*node*, *problem*), *frontier*)

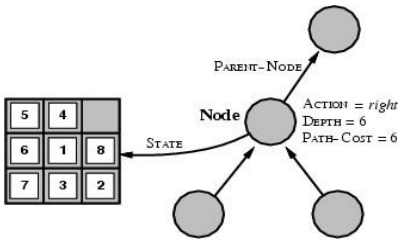
Determines search process!!

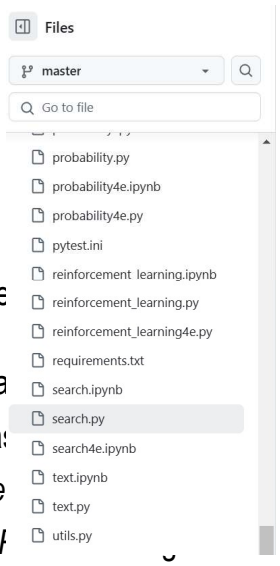
State space vs. search tree

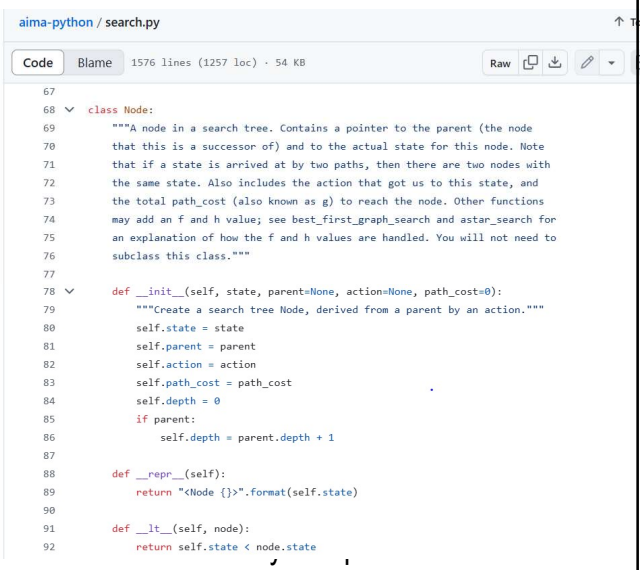


- A *state* is a (representation of) a physical configuration
- A *node* is a data structure belong to a search tree
  - A node has a parent, children, … and includes path cost, depth, …
  - Here *node*= <*state, parent-node, action, path-cost, depth*>
  - *FRONTIER* = contains generated nodes which are not yet expanded.
    - White nodes with black outline

State space vs. search tree

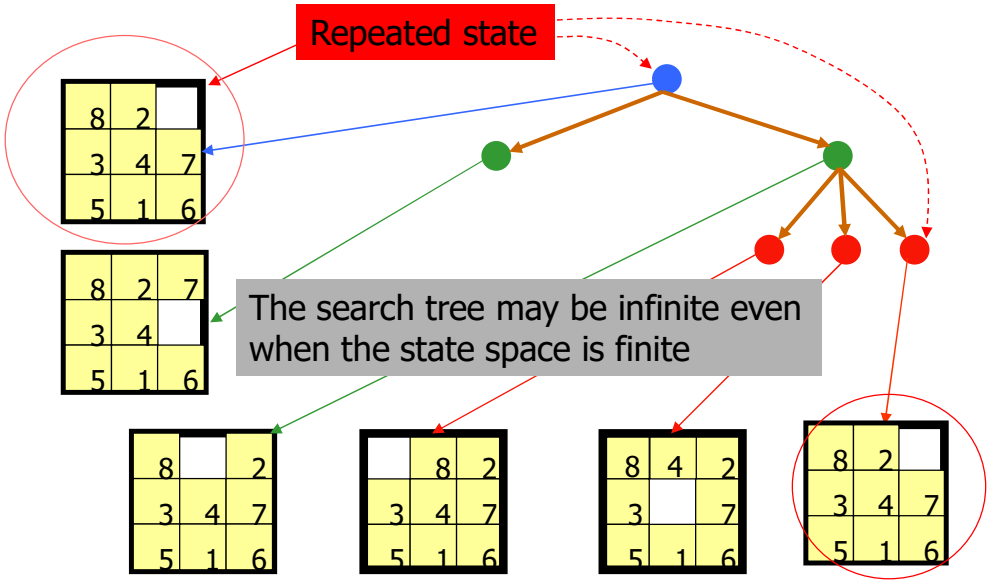






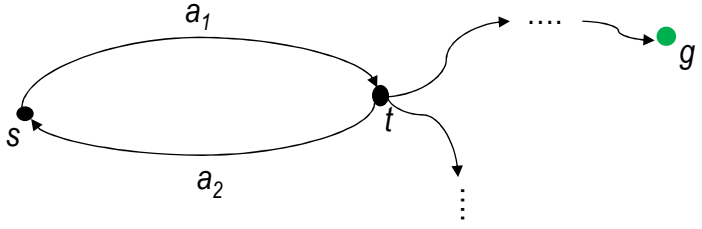
- A *state* is a (re
- A *node* is a da
  - A node has
  - Here *node*
  - *FRONTIER*
    - White nodes with black outline

Search Nodes ≠ States



Issues with repeated states (aka. Duplicate states)

- At best, repeated states mean that the same search will be performed over and over again
  - → Waste of space & time
- At worst, it can lead to an infinite loop
  - For example, the search will be bounced back and forth between states *s* and *t* and will never get to a goal state



Search algorithms – Eliminate repeated states

- Starts with a **FRONTIER** that contains the initial state
- Initialise an empty **EXPLORED** set
- Repeat:
  - If the **FRONTIER** happens to be empty, then *no solution*
  - Take a *node* out of the **FRONTIER** (this is *the option we choose to expand next*)
  - If that *node* contains a goal state, return the *found solution*
  - Add *node*.state to the **EXPLORED** set
  - Expand *node*, add the resulting nodes (from *node*) to the **FRONTIER** if their states have not already been in the **FRONTIER** or the **EXPLORED** set

Search algorithms – Eliminate repeated states

- Starts with a **FRONTIER** that contains the initial state
- Initialise an empty **EXPLORED** set
- Repeat:
  - If the **FRONTIER** happens to be empty, then *no solution*
  - Take a *node* out of the **FRONTIER** (this is *the option we choose to expand next*)
  - If that *node* contains a goal state, return the *found solution*
  - Add
  - Exp
    hav

Do pay attention to this statement because you may end up in a ***suboptimal solution***. WHY??

  - Expand *node*, add the resulting nodes (from *node*) to the **FRONTIER** if their states have not already been in the **FRONTIER** or the **EXPLORED** set

Search strategies

- A strategy is defined by picking the order of node expansion.
- Problem-solving performance is measured in four ways:
  - Completeness; *Does it always find a solution if one exists?*
  - Optimality; *Does it always find the least-cost solution?*
  - Time Complexity; *Number of nodes generated/expanded?*
  - Space Complexity; *Number of nodes stored in memory during search?*
- Time and space complexity are measured in terms of problem difficulty defined by:
  - *b - maximum branching factor of the search tree*
  - *d - depth of the least-cost solution*
  - *m - maximum depth of the state space (may be ∞)*

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Blind vs. Heuristic Strategies

- Blind (or uninformed) strategies do not exploit any of the information contained in a state
- Heuristic (or informed) strategies exploits such information to assess that one node is “more promising” than another

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Uninformed search strategies

- Categories defined by expansion algorithm:
  - ☑ Breadth-first search
  - ☑ Uniform-cost search
  - ☑ Depth-first search
  - ☐ Depth-limited search
  - ☐ Iterative deepening search.
  - ☐ Bidirectional search

Uninformed search strategies

- Breadth-first
  - ☐ Bidirectional
- Depth-first
  - ☐ Depth-limited
  - ☐ Iterative deepening

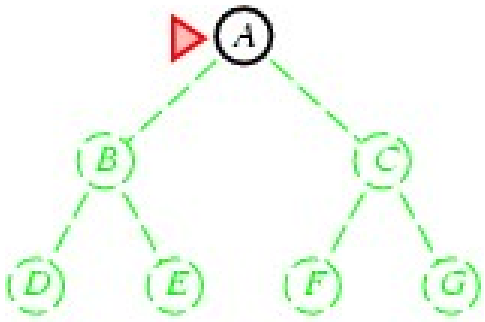
Step cost = 1

- Uniform-Cost
- (**= Dijkstra**)

Step cost = c(action) $\geq \varepsilon > 0$

Breadth-first search

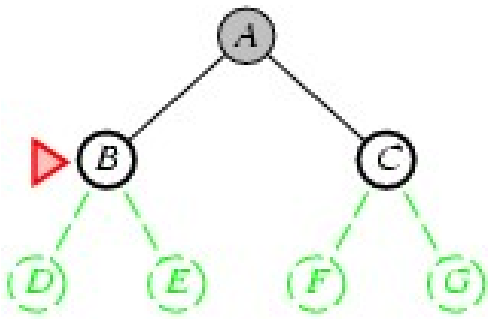
- Expand shallowest unexpanded node
- Implementation:
  - *frontier* is a FIFO queue, i.e., new successors go at end
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Breadth-first search

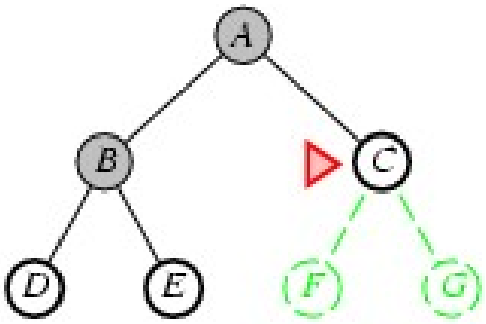
- Expand shallowest unexpanded node
- Implementation:
  - *frontier* is a FIFO queue, i.e., new successors go at end
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Breadth-first search

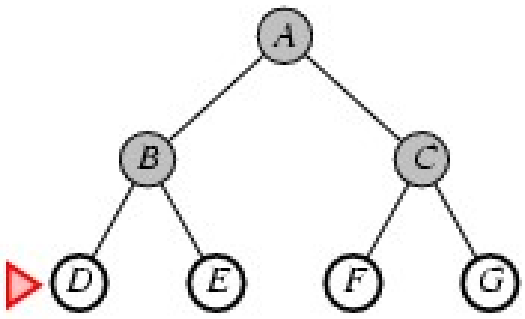
- Expand shallowest unexpanded node
- Implementation:
  - *frontier* is a FIFO queue, i.e., new successors go at end
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Breadth-first search

- Expand shallowest unexpanded node
- Implementation:
  - *frontier* is a FIFO queue, i.e., new successors go at end
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Breadth-first search

- Expan
- Imple
  - fro
  -

Files

master

Go to file

probability.py
probability4e.ipynb
probability4e.py
pytest.ini
reinforcement_learning.ipynb
reinforcement_learning.py
reinforcement_learning4e.py
requirements.txt
search.ipynb
search.py
search4e.ipynb
text.ipynb

aima-python / search.py ↑ Top

Code Blame 1576 lines (1257 loc) · 54 KB Raw

```
237
238 def breadth_first_graph_search(problem):
239     """[Figure 3.11]
240     Note that this function can be implemented in a
241     single line as below:
242     return graph_search(problem, FIFOQueue())
243     """
244     node = Node(problem.initial)
245     if problem.goal_test(node.state):
246         return node
247     frontier = deque([node])
248     explored = set()
249     while frontier:
250         node = frontier.popleft()
251         explored.add(node.state)
252         for child in node.expand(problem):
253             if child.state not in explored and child not in frontier:
254                 if problem.goal_test(child.state):
255                     return child
256                 frontier.append(child)
257     return None
258
```


Properties of breadth-first search

- Complete? Yes (if *b* is finite)
- Optimal? Yes (if cost = 1 per step)
-
- Time? $1+b+b^2+b^3+\ldots+b^d + b(b^d-1) = O(b^{d+1})$
- Space? $O(b^{d+1})$ (keeps every node in memory)
- Space is the bigger problem (more than time)
-

Time and Memory Requirements

d	#Nodes	Time	Memory
2	111	.01 msec	11 Kbytes
4	11,111	1 msec	1 Mbyte
6	~10^6	1 sec	100 Mb
8	~10^8	100 sec	10 Gbytes
10	~10^{10}	2.8 hours	1 Tbyte
12	~10^{12}	11.6 days	100 Tbytes
14	~10^{14}	3.2 years	10,000 Tb

Assumptions: b = 10; 1,000,000 nodes/sec; 100bytes/node

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Time and Memory Requirements

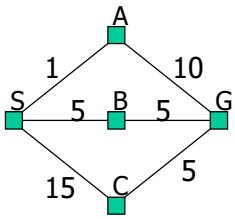
d	#Nodes	Time	Memory
2	111	.01 msec	11 Kbytes
4	11,111	1 msec	1 Mbyte
6	~10^6	1 sec	100 Mb
8	~10^8	100 sec	10 Gbytes
10	~10^{10}	2.8 hours	1 Tbyte
12	~10^{12}	11.6 days	100 Tbytes
14	~10^{14}	3.2 years	10,000 Tb

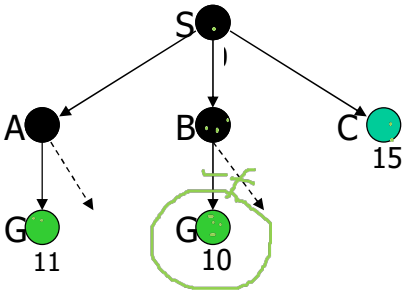
Assumptions: b = 10; 1,000,000 nodes/sec; 100bytes/node

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Uniform-Cost Strategy

- Each step has some cost $\geq \varepsilon > 0$.
- The cost of the path to each frontier node N is
  $g(N) = \Sigma$ costs of all steps.
- The goal is to generate a solution path of minimal cost.
- The queue FRONTIER is sorted in increasing cost.

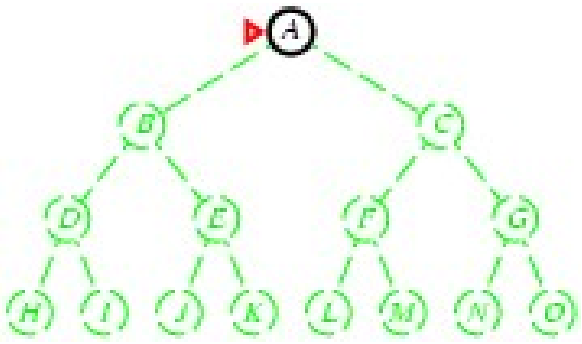




SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

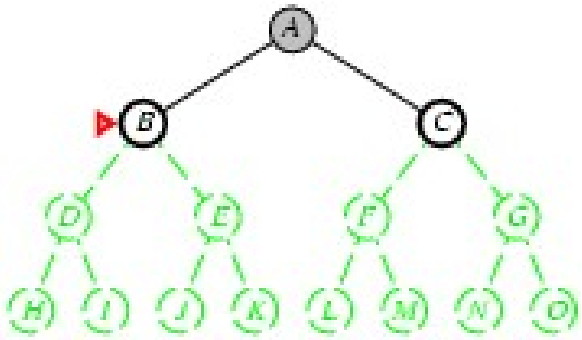
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

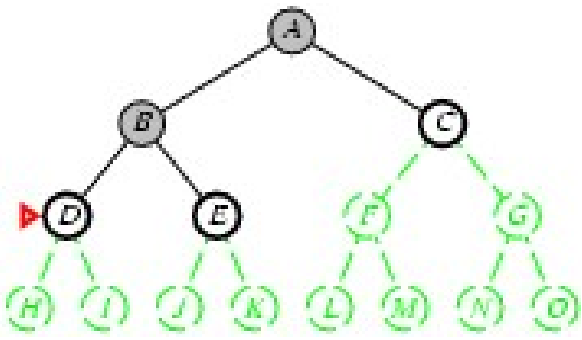
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

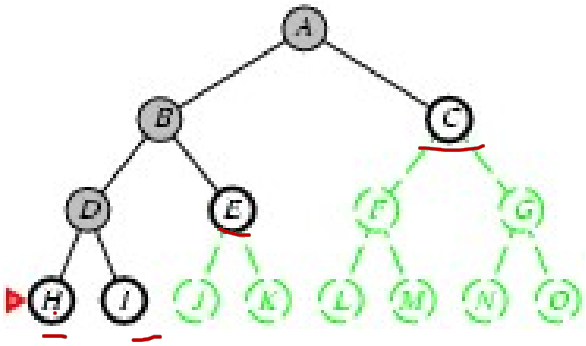
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

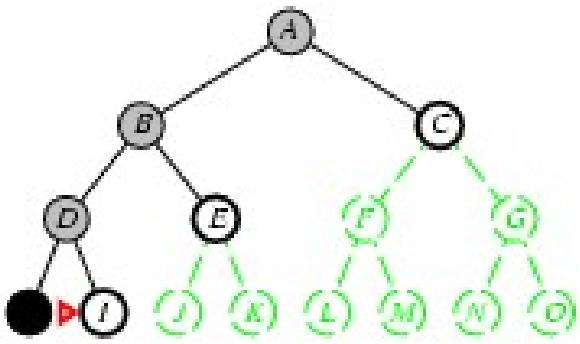
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

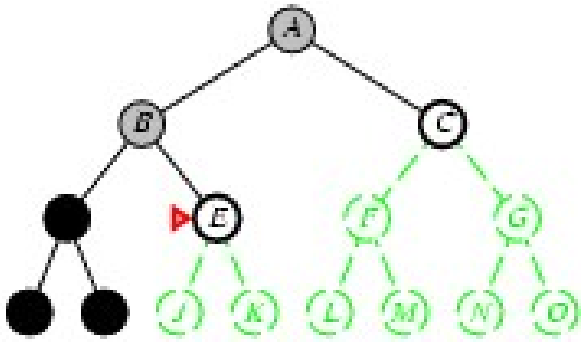
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

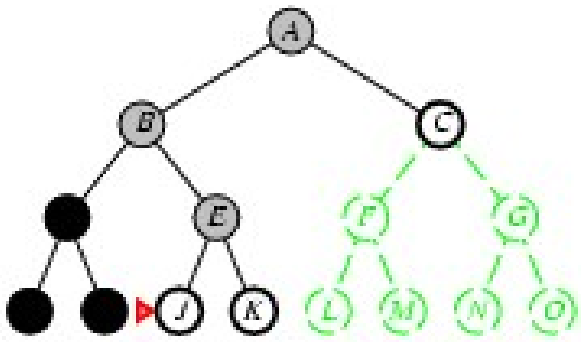
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

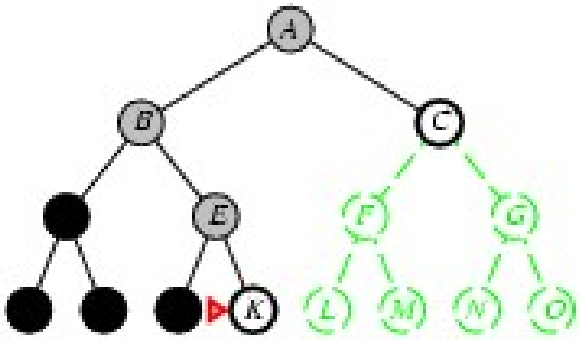
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

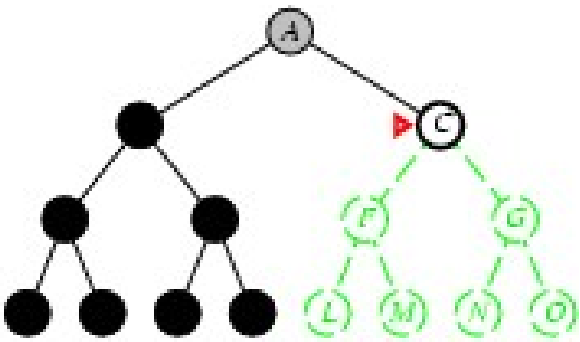
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

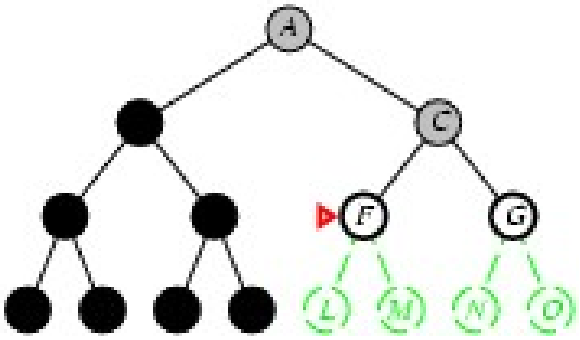
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

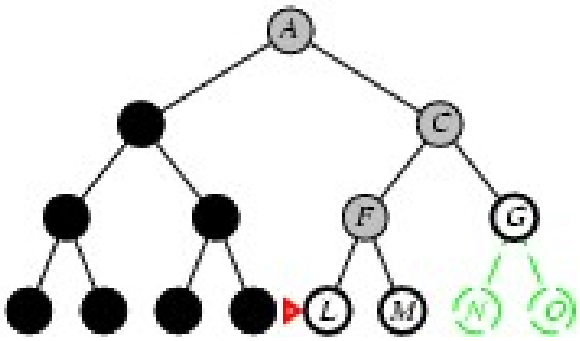
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

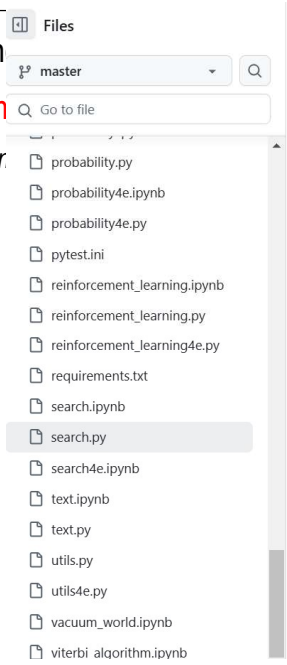
- Expand deepest unexpanded node
- Implementation:
  - *frontier* = LIFO queue, i.e., put successors at front
  -



SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Depth-first search

- Expan
- Implen
  - fror
  -



aima-python / search.py — Top

Code | Blame | 1576 lines (1257 loc) · 54 KB | Raw

```
196
197  def depth_first_tree_search(problem):
198      """
199      [Figure 3.7]
200      Search the deepest nodes in the search tree first.
201      Search through the successors of a problem to find a goal.
202      The argument frontier should be an empty queue.
203      Repeats infinitely in case of loops.
204      """
205
206      frontier = [Node(problem.initial)]  # Stack
207
208      while frontier:
209          node = frontier.pop()
210          if problem.goal_test(node.state):
211              return node
212          frontier.extend(node.expand(problem))
213      return None
214
215
216  def depth_first_graph_search(problem):
217      """
218      [Figure 3.7]
219      Search the deepest nodes in the search tree first.
220      Search through the successors of a problem to find a goal.
221      The argument frontier should be an empty queue.
222      Does not get trapped by loops.
223      If two paths reach a state, only use the first one.
224      """
225      frontier = [(Node(problem.initial))]  # Stack
```

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
  - Modify to avoid repeated states along path
    - → complete in finite spaces
- Optimal? No
-
- Time? $O(b^m)$: terrible if m is much larger than d
  - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Summary

- Search tree ≠ state space
- Search strategies: breadth-first, depth-first, and variants
- Evaluation of strategies: completeness, optimality, time and space complexity
- Avoiding repeated states
- Optimal search with variable step costs

SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH